# Case Study 1: Building a Mean-Variance Portfolio Pipeline

**?var:course.code – ?var:course.title**

Due: see Gradescope

## Overview

**The scenario:** You've just joined the quantitative strategies group at a mid-size asset manager. Your portfolio manager wants a sector-rotation pipeline built on mean-variance theory — but before any capital gets allocated, she wants proof that you understand *how much to trust your own numbers*. That means a reproducible data pipeline, an honest accounting of estimation and model uncertainty, a sensitivity analysis of the optimizer's outputs, and a backtest that reflects real trading frictions rather than a frictionless fantasy. This is, in miniature, the actual workflow of a junior quant researcher — and the structure of this case study (build → interrogate the inputs → stress-test the outputs → backtest honestly) mirrors take-home exercises used in real quant research interviews.

In modules 1 and 2 you learned the building blocks of mean-variance portfolio theory: how to compute estimates of forecast moments $\mu$ and $\Sigma$, how the optimal weight vector $\mathbf{w}^*$ follows from those moments and a risk-aversion parameter $\gamma$, and how cross-sectional regression (CAPM) provides a structured way to estimate those moments. This case study asks you to turn those ideas into a working pipeline applied to a real universe of assets.

The universe is the eleven SPDR sector ETFs — **XLC, XLY, XLP, XLE, XLF, XLV, XLI, XLB, XLRE, XLK, XLU** — plus the S&P 500 index ETF **SPY** as the market factor.

Submit your work as a single, self-contained **Jupyter notebook** (`.ipynb`). Every section should be clearly delineated with a markdown cell. Code must run top-to-bottom without error. Figures must render inline. Show all intermediate outputs that support your written interpretation.

### Some Rookie Mistakes That Will Sink a Real Portfolio

Every one of these has ended a junior quant's strategy before it ever got funded. Watch for them:

- **Lookahead bias / data leakage.** Using anything from the test period — even implicitly, like fitting a scaler or picking a hyperparameter — to make decisions in the training period. If you ever "know" something in Section 5's loop that you couldn't have known on that actual trading day, you have leakage.
- **Off-by-one errors in the trading loop.** Estimating $\hat{\mu}_t$ and $\hat{\Sigma}_t$ using data *through* day $t$ and then trading on day $t$'s close (rather than day $t+1$) silently injects lookahead bias into a walk-forward backtest — this is the single most common bug in Section 5.
- **Mixing return conventions.** Switching between log returns and simple (percent) returns partway through a notebook, or forgetting that log returns don't sum across assets the way simple returns do when computing portfolio returns.
- **Annualization errors.** Multiplying mean daily return by 252 but volatility by 252 instead of $\sqrt{252}$ (or vice versa), or comparing a Sharpe ratio computed on daily data to one computed on monthly data without adjusting.
- **Treating unconstrained weights as "the answer."** If $\mathbf{w}^*$ wants to short one sector 300% and lever another 400%, that's a signal your inputs are noisy or your $\gamma$ is too low — not a strategy to trade as-is (see Sections 3 and 3.5).
- **Reporting gross returns dressed up as net returns**, or applying transaction costs at the wrong point in the loop (e.g., after applying the return instead of before, which under- or overstates P&L).
- **Silent stale-cache bugs.** A cache that never re-validates against the latest available trading day will quietly grade you on old data — and a grader re-running your notebook weeks later may get different results than you did.
- **Cherry-picking the flattering metric.** Reporting Sharpe ratio and cumulative return but conveniently leaving out max drawdown or a bad three-month stretch. A real PM will ask about the drawdown before anything else.

---

## Section 1 — Data Pipeline

*In practice: this is the unglamorous infrastructure work every quant desk depends on — nobody trusts a signal built on data nobody can reproduce.*

Write a function (or small collection of functions) that fetches daily adjusted closing prices for the twelve tickers above using `yfinance`. The function must implement a **disk cache**: on the first call it downloads the data and saves it to a local file (e.g., a CSV or Parquet); on subsequent calls it reads from disk and only queries the internet if the locally stored data does not cover the most recent available trading day.

The pipeline should return a clean `DataFrame` of **daily log returns** (or percent returns — be explicit about which you choose and be consistent throughout the notebook), with dates as the index and tickers as columns, with `NaN` rows dropped.

Your cache should behave correctly across different machines and be easy for a grader to reproduce. Include a brief prose cell explaining your cache invalidation logic.

---

## Section 2 — Parameter and Model Uncertainty

*In practice: this is what risk and model-validation teams call "parameter risk" or "model risk" — before a strategy ever touches real capital, someone has to answer "how much do we actually trust these inputs?"*

The central inputs to portfolio optimization are (estimates of) $\mu$ and $\Sigma$. Before we ever optimize, we should ask: **how well can we actually estimate these?**

### 2.1 — Simulation Study (No Model Risk)

Suppose the *true* return distribution is multivariate normal with some known $\mu^*$ and $\Sigma^*$ (use your full-sample estimates from the sector ETF data as the "ground truth"). For sample sizes $n \in \{63, 126, 252, 504, 1008, 2520\}$ (roughly one quarter, one half year, one year, two years, four years, and ten years of trading days):

- Simulate many independent samples of size $n$ from $\mathcal{N}(\mu^*, \Sigma^*)$.
- For each sample, compute the sample mean $\hat{\mu}$ and sample covariance $\hat{\Sigma}$.
- Summarize, across simulations, the estimation error for both $\mu$ and $\Sigma$.

Produce visualizations that make a clear and honest case for how reliable the estimates are at each $n$. Some quantities you might consider: element-wise RMSE, the Frobenius norm of $\hat{\Sigma} - \Sigma^*$, the distribution of $\hat{\mu}_i - \mu_i^*$ for a few assets, or the distribution of estimated Sharpe ratios for a fixed $\mathbf{w}$. You are encouraged to go beyond these suggestions. Write a brief interpretation: at what sample sizes are the estimates "good enough" for practical use, and are $\mu$ and $\Sigma$ equally reliable?

### 2.2 — Model Uncertainty on Real Data

Now use the actual sector ETF returns. Estimate $\mu$ and $\Sigma$ under at least two different approaches:

- **Sample moments**: sample mean vector and sample covariance matrix.
- **CAPM**: multivariate OLS with SPY as the single factor (as derived in module 2), then use the law of total expectation and law of total covariance to obtain $\hat{\mu}$ and $\hat{\Sigma}$.

For each pair of estimates, report the full mean vector and covariance matrix (or correlation matrix — your choice). How different are the estimates across methods? Which elements differ the most? What does that tell you about model sensitivity?

### 2.3 — Bootstrap and Rolling Window

Choose one estimation method from 2.2. Using the same data:

- **Bootstrap**: resample the daily return rows with replacement 500–1000 times. For each resample, compute $\hat{\mu}$ and $\hat{\Sigma}$. Visualize the resulting distributions. For instance: plot the bootstrap distribution of $\hat{\mu}_i$ for each sector with a confidence band; plot the distribution of $\hat{\Sigma}_{ij}$ for a selection of covariance pairs.
- **Rolling window**: compute $\hat{\mu}$ and $\hat{\Sigma}$ using a rolling 252-day window. Plot how these estimates evolve over time. Are there identifiable regimes? Do any estimates look stable, or are they volatile?

Write at least one paragraph interpreting what you observe, with specific reference to your figures.

### 2.4 — Dimension vs. Sample Size: When the Universe Outgrows the Data

Sections 2.1–2.3 held the universe fixed at 12 assets and varied how much *history* you had. But a PM can just as easily expand the *universe* — trading 50 or 200 individual names instead of 12 sector ETFs — without getting any more years of data to estimate with. This subsection asks the same question as 2.1 (how good is $\hat{\Sigma}$?), but sweeps dimension $k$ instead of sample size $n$.

Fix the sample size at $n = 252$ (one trading year — a realistic amount of history for a newer strategy or a recently-listed set of names) for the entire subsection. You'll need a "ground truth" $\Sigma^*(k)$ that scales to arbitrary $k$, since you only have 12 real assets to work with:

- For $k \leq 12$, you can use your real full-sample sector-ETF estimates (as in 2.1), subset to the first $k$ assets.
- For $k > 12$, construct a synthetic ground truth with the same one-factor structure as your CAPM estimates in 2.2: $\Sigma^*(k) = \mathbf{D} + \beta\beta^\top \sigma_m^2$, where $\sigma_m^2$ is your estimated market (SPY) variance, and the $k$-vector $\beta$ and diagonal idiosyncratic variances $\mathbf{D}$ are drawn to match the *empirical spread* of the betas and idiosyncratic variances you actually estimated for the 12 sector ETFs in 2.2 (e.g., sample from a distribution fit to those 12 real values, rather than inventing arbitrary numbers). Document this construction in a prose cell — the point isn't that this is a realistic 200-asset universe, it's that it's a $k$-asset ground truth grounded in real, estimated parameters.

For a grid of $k$ that spans well below, near, and **above** $n = 252$ (e.g., $k \in \{5, 12, 50, 150, 252, 400\}$), simulate many samples of size $n$ from $\mathcal{N}(\mu^*(k), \Sigma^*(k))$ and compute $\hat{\Sigma}$ for each. Track, across simulations and as a function of $k$:

- The **condition number** of $\hat{\Sigma}$ (ratio of largest to smallest eigenvalue) — a numerically well-behaved matrix has a modest condition number; a nearly-singular one has a huge one.
- The **smallest eigenvalue** of $\hat{\Sigma}$ directly. What happens to it as $k$ approaches, and then exceeds, $n$? (Recall that a sample covariance matrix estimated from $n$ observations of $k$ variables is mathematically guaranteed to be singular once $k > n$ — this isn't a modeling failure, it's a hard fact about how many independent directions of variation $n$ data points can possibly pin down.)
- The magnitude of the resulting **unconstrained** $\mathbf{w}^*$ (e.g., its $L_2$ norm, or gross leverage $\sum_i |w_i|$) computed from each simulated $\hat{\Sigma}$, at a fixed $\gamma$.

Produce a plot (or small panel of plots) showing all three quantities against $k$, on a log scale for $k$ if that helps readability. Write a paragraph connecting what you see to the closed-form $\mathbf{w}^* \propto \hat{\Sigma}^{-1}\hat{\mu}$: why does a near-singular (or literally singular) $\hat{\Sigma}$ make the *inverse* — and therefore the weights — blow up numerically, even when the *true* $\Sigma^*(k)$ is perfectly well-behaved at every $k$? This is the dimension-side counterpart to 2.1's sample-size finding, and it's the reason Section 3.5's constrained weights and (in industry) shrinkage estimators like Ledoit-Wolf exist — not required here, but worth knowing that "regularize $\hat{\Sigma}$ before inverting it" is a standard professional response to exactly the failure mode you just produced.

---

## Section 3 — How Uncertainty in Parameters Propagates to Weights

*In practice: this is a sensitivity / stress-testing exercise — the kind of analysis a risk team runs on any model before it's approved, and the kind of question a PM will ask you the moment you present a portfolio: "how fragile is this?"*

The quantities $\mu$ and $\Sigma$ are not directly observable in a portfolio — the observable outcome is the weight vector $\mathbf{w}^*$. This section asks you to understand *how sensitive* $\mathbf{w}^*$ is to its inputs.

Use a fixed base estimate (e.g., your CAPM or sample-moment estimates from Section 2 on the full training window) as the reference $\hat{\mu}$ and $\hat{\Sigma}$.

### 3.1 — Sensitivity to $\gamma$

Compute $\mathbf{w}^*(\gamma)$ for a grid of risk-aversion values, say $\gamma \in [0.1, 20]$ on a log scale. For each $\gamma$, record the full weight vector. Produce a plot that shows how each sector's weight evolves across

this range. What are the extreme cases? At what $\gamma$ does the portfolio become "reasonable" (no extreme allocations)?

### 3.2 — Sensitivity to $\mu$

Starting from the base $\hat{\mu}$, explore what happens as you move it toward zero. Specifically, define the family

$$\mu(\alpha) = (1 - \alpha)\,\hat{\mu}, \qquad \alpha \in [0, 1].$$

At $\alpha = 0$ you have the estimated means; at $\alpha = 1$ all means are zero. Compute $\mathbf{w}^*$ along this path (fixing $\hat{\Sigma}$ and a chosen $\gamma$) and visualize how the weights evolve. At what value of $\alpha$ do the weights become "well-behaved"?

### 3.3 — Sensitivity to Correlations

To inflate or deflate correlations while keeping the matrix positive definite and the variances unchanged, you need to work with the **correlation matrix**. Decompose:

$$\Sigma = \mathbf{D}\,\mathbf{C}\,\mathbf{D},$$

where $\mathbf{D} = \mathrm{diag}(\sigma_1, \ldots, \sigma_k)$ is the diagonal matrix of standard deviations and $\mathbf{C}$ is the correlation matrix. Define the perturbed correlation matrix

$$\mathbf{C}(\rho) = (1 - \rho)\,\mathbf{C} + \rho\,\mathbf{1}\mathbf{1}^\top / k, \qquad \rho \in [0, 1),$$

(or another reasonable perturbation of your choice) and reconstruct $\Sigma(\rho) = \mathbf{D}\,\mathbf{C}(\rho)\,\mathbf{D}$. Compute $\mathbf{w}^*$ across values of $\rho$ and visualize. What is the intuition for what happens as all assets become more correlated?

### 3.4 — Sensitivity to Volatility

Using the same decomposition $\Sigma = \mathbf{D}\,\mathbf{C}\,\mathbf{D}$, define the scaled covariance matrix

$$\Sigma(s) = (s\,\mathbf{D})\,\mathbf{C}\,(s\,\mathbf{D}), \qquad s > 0,$$

which scales all standard deviations by $s$ while preserving all pairwise correlations. Compute $\mathbf{w}^*$ across values of $s \in [0.5, 3]$ and visualize. Does scaling volatility up or down change the *composition* of the portfolio or just the implied risk?

### 3.5 — A Dose of Realism: Constrained Weights

The closed-form $\mathbf{w}^*$ you've been computing is unconstrained — it is free to short assets and to take arbitrarily large positions. Real mandates almost always impose some constraint on top of the mean-variance objective, though which constraint depends a lot on the fund. A few common ones:

- **Long-only** ($w_i \geq 0$): common for retail-facing funds and many institutional mandates, but plenty of funds (long/short equity, market-neutral, macro) short freely — this is a mandate choice, not a law of nature.
- **Position cap** (e.g., $w_i \leq 0.25$): limits concentration in any single name or sector.
- **Leverage constraint** ($\sum_i |w_i| \leq L$ for some $L$, e.g. $L = 1$ for no leverage or $L = 2$ for 2-to-1 gross leverage): caps total gross exposure regardless of the long/short split — this is how many hedge funds actually bound risk instead of (or in addition to) going long-only.

Pick **one or two constraints from this list (or a reasonable variant of your own)** that you find most interesting, and re-solve for $\mathbf{w}^*$ under them using your base $\hat{\mu}$ and $\hat{\Sigma}$ from Section 2. You will need a numerical solver rather than the closed-form formula — `scipy.optimize.minimize` (SLSQP) or `cvxpy` both work well for this (note the leverage constraint is not smooth as written; either reformulate it for your solver or pick a smooth relaxation). Compare the constrained weights to the unconstrained weights from earlier in Section 3: what changed, and how much expected utility (in the mean-variance objective) is "given up" by imposing the constraint(s) you chose?

---

## Section 4 — The Effect of Rebalancing and Trading Costs

*In practice: this is implementation shortfall — the gap between a paper strategy and a tradeable one. It's often the difference between a strategy that gets funded and one that doesn't.*

Sections 2 and 3 were about the inputs. This section is about what happens when we actually use $\mathbf{w}^*$ in practice.

**Train/test split**: use the first 60% of your data as the training period (estimate parameters here) and the remaining 40% as the test period (evaluate performance here). Never use test data to make portfolio decisions.

**Benchmarks**: in every comparison below, include two passive benchmarks: - **Equal-weight (EW)**: $\mathbf{w} = \mathbf{1}/k$ held fixed and rebalanced each period. - **SPY**: the market ETF return series.

### 4.1 — Frictionless Rebalancing

Assume you can trade at zero cost. At the start of the test period, invest according to $\mathbf{w}^*$ computed on the training data. Rebalance back to $\mathbf{w}^*$ every trading day (i.e., $\mathbf{w}$ is held constant throughout the test period).

Report and plot:

- Cumulative return over the test period (log scale optional)
- Annualized Sharpe ratio (use the risk-free rate from one of your data sources, or set it to zero and note that)
- Maximum drawdown
- Annualized volatility

How do your model-based portfolios compare to the benchmarks?

### 4.2 — Introducing Transaction Costs

Now make the simulation more realistic. At each rebalancing event, you must buy and sell assets to restore the target weights. Each dollar of turnover incurs a cost of $c$ (expressed in basis points, where $1\,\text{bp} = 0.0001$). The turnover at time $t$ is:

$$\text{turnover}_t = \sum_{i=1}^{k} |w_{t,i}^{\text{target}} - w_{t,i}^{\text{drifted}}|$$

where $w_{t,i}^{\text{drifted}}$ is the weight in asset $i$ just *before* rebalancing, after price changes have moved the portfolio away from the target. The cost is deducted from the portfolio value before applying the next period's returns.

Additionally, model a fixed **management fee** of $f$ basis points per year, deducted as a daily haircut: $r_{\text{net},t} = r_t - f/252$.

Investigate performance as a function of:

- Rebalancing frequency (daily, weekly, monthly, quarterly)
- Cost level $c$ (try a few values in the range 1–20 bps)

Produce at least one plot of net-of-cost annualized Sharpe ratio as a function of rebalancing frequency and cost level. What rebalancing frequency is approximately optimal? Does the answer depend on $c$?

---

## Section 5 — Full Backtester

*In practice: a walk-forward backtest is the minimum bar for a strategy to be taken seriously by any PM or allocator — and this is close to verbatim what many quant research take-home interviews ask you to build. Do this section well and you'll have a legitimate work sample.*

Sections 1–4 used a single fixed train/test split. This section asks you to build a proper **walk-forward backtester** that simulates how a real portfolio manager would operate: at each point in time, use only the history available up to that point to estimate parameters and set weights.

### 5.1 — The Loop

Implement the following loop. Let $T$ be the full length of your return series and $T_0$ be a minimum burn-in period (e.g., 252 trading days — one year of history).

For $t = T_0, T_0 + 1, \ldots, T - 1$:

1. **Fit**: using only returns from day 1 through day $t$ (expanding window) or a rolling window of fixed length $W$, estimate $\hat{\mu}_t$ and $\hat{\Sigma}_t$.
2. **Optimize**: compute $\mathbf{w}_t^* = \mathbf{w}^*(\hat{\mu}_t, \hat{\Sigma}_t, \gamma)$ using the MPT formula.
3. **Trade**: if $t$ is a rebalancing day (e.g., the first trading day of each month), execute trades to move from the current drifted weights to $\mathbf{w}_t^*$. Apply transaction costs as in Section 4.2.
4. **Evolve**: apply day $t + 1$ returns to the portfolio to get the new dollar value and drifted weights.
5. **Record**: store $r_{t+1}^{\text{net}}$, $\mathbf{w}_t^*$, $\text{turnover}_t$, and $\text{costs}_t$.

You must use either an expanding window or a rolling window — justify your choice in a prose cell.

### 5.2 — Performance Report: The Tearsheet

Collect the time series of out-of-sample net returns produced by the loop and package your results as a **one-page tearsheet** — the kind of summary a PM would actually be handed before deciding whether to allocate capital, in the style of tools like `quantstats` or `pyfolio` (you are welcome to use either package, or build your own).

The tearsheet should combine, on a single logical "page" (a markdown cell plus a compact figure or two is fine):

| Metric | Value |
|---|---|
| Annualized return | |
| Annualized volatility | |

| Metric | Value |
|---|---|
| Sharpe ratio | |
| Maximum drawdown | |
| Total turnover (annualized) | |
| Total costs paid (% of initial capital) | |

Plot:

- Cumulative wealth of your strategy vs. the SPY and equal-weight benchmarks
- Monthly turnover over time
- A heatmap or line chart of portfolio weights over time

Close the tearsheet with a short **PM-facing recommendation paragraph** (3-5 sentences, written as if addressed to a portfolio manager deciding whether to fund this strategy): does the evidence support allocating capital to it, and under what conditions or caveats?

### 5.3 — Interpretation

Write a paragraph (or two) addressing each of the following:

- Does the walk-forward strategy beat the benchmarks? If yes, where does the outperformance come from? If no, why not?
- How did transaction costs change your conclusion relative to the frictionless result in Section 4.1?
- What did the weight time series reveal? Did the optimizer concentrate in a small number of sectors, or did it diversify? Was this stable over time?
- If you were managing this portfolio with real money, what changes would you make? Point specifically to findings from Sections 2 and 3 to support your answer.

---

## Section 6 — Bonus: Push the Strategy Further (Optional, up to 5 extra points)

Everything above gets you a defensible, honestly-evaluated strategy. If you want to see how it stacks up against other approaches a real desk might consider, pick **one** of the following and add it to your backtest comparison:

- A **minimum-variance** portfolio (ignore $\mu$ entirely and minimize risk) as an additional benchmark.
- A **risk-parity** portfolio (weights inversely proportional to volatility, or the full risk-parity solution) as an additional benchmark.

- A simple **momentum overlay**: tilt $\hat{\mu}$ toward recent trailing returns before optimizing, and see whether it helps or hurts out-of-sample.

Briefly explain your choice and what it revealed relative to your Section 5 strategy. This section is not required for full credit on the assignment — it's here for students who want to keep exploring.

---

## Deliverables and Grading

Submit a single `.ipynb` that runs top-to-bottom without error. You do not need to commit your cached data file; include the caching function so that a grader can reproduce the download. Where sections ask for "good visualizations," axis labels, legends, and titles are required. Unlabeled plots will not receive full credit.

| Section | Points |
|---|---|
| 1 — Data pipeline (caching, clean returns) | 10 |
| 2 — Parameter uncertainty (simulation + real data + bootstrap/rolling + dimension) | 30 |
| 3 — Weight sensitivity (all five sub-parts + visualizations) | 20 |
| 4 — Rebalancing and costs (frictionless + cost analysis) | 20 |
| 5 — Backtester (loop + tearsheet + interpretation) | 20 |
| 6 — Bonus | up to +5 |
| **Total** | **100 (+5 bonus)** |

Partial credit is given for each section independently. A section that is attempted but incomplete receives more credit than a section that is skipped.

**A note on why this matters beyond the grade:** a cleaned-up version of this notebook — especially Sections 5 and 6 — is a legitimate portfolio piece. Many quant research take-home interviews ask for almost exactly this: a reproducible data pipeline, an honest treatment of uncertainty, and a walk-forward backtest with costs. Consider putting a polished version on GitHub.